📚 README PRINCIPAL - OpenTalkWisp

CRM Omnicanal con Inteligencia Artificial para WhatsApp

Plataforma SaaS de gestión de relaciones con clientes que centraliza comunicaciones de WhatsApp, Facebook e Instagram con automatización inteligente mediante IA.

🚀 Demo • 📖 Documentación • 🐛 Reportar Bug • 💡 Solicitar Feature

---

📋 Tabla de Contenidos

- 🎯 ¿Qué es OpenTalkWisp?
- ✨ Características Principales
- 🏗️ Arquitectura del Sistema
- 🚀 Inicio Rápido
- 🛠️ Stack Tecnológico
- 📁 Estructura del Proyecto
- 🔧 Configuración
- 🤖 Integración de IA
- 💬 Proveedores de WhatsApp
- 📊 Base de Datos
- 🧪 Testing
- 🚢 Despliegue
- 📚 Documentación Completa

- ☐ Contribuir
- 📄 Licencia

🎯 ¿Qué es OpenTalkWisp?

OpenTalkWisp es un CRM moderno y completo diseñado específicamente para empresas que necesitan gestionar comunicaciones masivas a través de WhatsApp y otras plataformas de mensajería, potenciado por Inteligencia Artificial.

🏪 Problema que Resuelve

Las empresas enfrentan desafíos al gestionar múltiples conversaciones en WhatsApp:

- ❌ Pérdida de mensajes importantes
- ❌ Falta de seguimiento de leads
- ❌ Respuestas lentas o inconsistentes
- ❌ Dificultad para escalar atención al cliente
- ❌ Sin métricas ni análisis de conversaciones

✅ Solución OpenTalkWisp

- ✅ **Centralización**: Todos los mensajes en una sola plataforma
- ✅ **Automatización**: Respuestas inteligentes con IA
- ✅ **CRM Integrado**: Gestión completa de leads y ventas
- ✅ **Omnicanalidad**: WhatsApp, Facebook, Instagram en un solo lugar
- ✅ **Analytics**: Métricas y reportes en tiempo real
- ✅ **Escalabilidad**: Desde startups hasta empresas grandes

✨ Características Principales

💬 Gestión de Conversaciones

- **Bandeja Unificada**: Todos los chats de WhatsApp, Facebook e Instagram en un solo lugar

- **Asignación Inteligente**: Distribuye conversaciones automáticamente entre agentes
- **Respuestas Rápidas**: Plantillas y atajos para responder más rápido
- **Historial Completo**: Acceso a todo el historial de interacciones con cada contacto
- **Notas Internas**: Colaboración entre agentes con notas privadas
- **Estados de Conversación**: Abierta, En espera, Resuelta, Cerrada

☐ Inteligencia Artificial

- **Respuestas Automáticas**: IA genera respuestas contextuales basadas en el historial
- **Análisis de Sentimiento**: Detecta si el cliente está satisfecho, neutral o molesto
- **Lead Scoring Automático**: Califica leads del 0-100 según comportamiento e interacciones
- **Chatbots Personalizados**: Crea flujos conversacionales con lenguaje natural
- **Sugerencias para Agentes**: IA sugiere respuestas a los agentes humanos
- **Resumen de Conversaciones**: Resume conversaciones largas automáticamente
- **Extracción de Datos**: Identifica nombres, emails, teléfonos automáticamente

📊 CRM y Pipeline de Ventas

- **Gestión de Contactos**: Base de datos completa con tags, segmentación y campos personalizados
- **Pipeline Visual**: Tablero Kanban para mover leads por etapas de venta
- **Deals/Oportunidades**: Seguimiento de oportunidades de venta con valores y probabilidades
- **Actividades**: Log completo de llamadas, reuniones, emails y tareas
- **Importación Masiva**: Importa contactos desde CSV/Excel
- **Deduplicación**: Detecta y fusiona contactos duplicados automáticamente

📢 Campañas de Marketing

- **Broadcasting Masivo**: Envía mensajes a miles de contactos
- **Segmentación Avanzada**: Filtra por tags, estado de lead, score, etc.
- **Programación**: Programa campañas para fechas y horas específicas
- **Plantillas Multimedia**: Envía texto, imágenes, videos, documentos
- **Rate Limiting Inteligente**: Evita ser bloqueado por WhatsApp
- **Métricas en Tiempo Real**: Enviados, entregados, leídos, respuestas

🔗 Conexión Híbrida WhatsApp

- **Meta Cloud API**: Conexión oficial de WhatsApp Business
- **Conexión QR (Baileys)**: Conecta cualquier número sin API oficial
- **Multi-Instancia**: Gestiona múltiples números de WhatsApp
- **Patrón Adapter**: Cambia entre proveedores sin afectar el código

🔄 Flujos Conversacionales

- **Constructor Visual**: Crea flujos con drag & drop
- **Generación con IA**: Describe el flujo en texto y la IA lo crea
- **Nodos Avanzados**: Mensajes, preguntas, condiciones, acciones, webhooks, delays
- **Memoria Contextual**: Los flujos recuerdan el contexto de la conversación
- **Testing Integrado**: Prueba flujos antes de activarlos

📈 Analytics y Reportes

- **Dashboard Ejecutivo**: Métricas clave en tiempo real
- **Reportes de Agentes**: Rendimiento individual de cada agente
- **Análisis de Conversiones**: Tasa de conversión por etapa del pipeline
- **Métricas de Campañas**: ROI, tasa de apertura, tasa de respuesta
- **Exportación**: Descarga reportes en PDF, Excel, CSV

🔌 Integraciones y API

- **API REST Completa**: Integra con cualquier sistema externo
- **Webhooks**: Recibe eventos en tiempo real
- **Zapier/Make Compatible**: Conecta con 5000+ aplicaciones
- **SDK JavaScript/Python**: Librerías oficiales para desarrolladores

---

🏗️ Arquitectura del Sistema

Diagrama de Alto Nivel

Copiar

```
┌──────────────────────────────────────────────────────────────────┐
│                              FRONTEND                            │
│                   Next.js 14 + React + Tailwind                  │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐    │
```

```
│  Dashboard │  │  Chats  │  │ Pipeline │  Campaigns │  │
└────────────┘  └─────────┘  └──────────┘  └──────────┘  │
└──────────────────────────────────────────────────────────┘
                          │ REST API + WebSockets
                          ▼
┌──────────────────────────────────────────────────────────┐
│                         BACKEND                          │
│                  NestJS + TypeScript                     │
│  ┌──────────────────────────────────────────────────────┐│
│  │              API LAYER (Controllers)                 │
│  └──────────────────────────────────────────────────────┘│
│                          │                               │
│  ┌──────────────────────▼───────────────────────────────┐│
│  │           SERVICE LAYER (Business Logic)             │
│  │  • Auth   • Contacts   • Conversations   • Campaigns │
│  │  • AI     • Flows      • Pipeline        • Analytics │
│  └──────────────────────────────────────────────────────┘│
│                          │                               │
│  ┌──────────────────────▼───────────────────────────────┐│
│  │         WHATSAPP PROVIDER LAYER (Adapters)           │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐│
│  │  │ Meta Cloud   │  │   Baileys    │  │  WWebJS      │
│  │  │   Provider   │  │   Provider   │  │  Provider    │
│  │  └──────────────┘  └──────────────┘  └──────────────┘│
│  └──────────────────────────────────────────────────────┘│
└──────────────────────────────────────────────────────────┘
                          │
        ┌─────────────────┴─────────────────┬─────────────┐
        │                 │                 │             │
        ▼                 ▼                 ▼             ▼
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ PostgreSQL   │  │   Redis      │  │   OpenAI     │  │  Pinecone    │
│  (Prisma)    │  │   Cache      │  │  LangChain   │  │  Vectors     │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘
```

Patrón de Arquitectura

Clean Architecture + Adapter Pattern + Repository Pattern

- **Capa de Presentación**: Next.js (Frontend)
- **Capa de Aplicación**: Controllers y DTOs (NestJS)
- **Capa de Dominio**: Lógica de negocio y entidades
- **Capa de Infraestructura**: Adaptadores, DB, APIs externas

Flujo de un Mensaje

Copiar

```
WhatsApp → Webhook → Provider Adapter → Unified Message →
→ Business Logic → AI Analysis (opcional) → Database →
→ WebSocket → Frontend → Agent
```

🚀 Inicio Rápido

Prerequisitos

Asegúrate de tener instalado:

- **Node.js** >= 20.0.0 (Descargar)
- **pnpm** >= 8.0.0 (`npm install -g pnpm`)
- **Docker** y **Docker Compose** (Descargar)
- **Git** (Descargar)

Instalación en 5 Minutos

Copiar

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/opentalkwisp.git
cd opentalkwisp

# 2. Instalar dependencias
pnpm install

# 3. Copiar variables de entorno
cp .env.example .env

# 4. Editar .env con tus credenciales
nano .env  # o usa tu editor favorito

# 5. Levantar base de datos con Docker
docker-compose up -d

# 6. Ejecutar migraciones de base de datos
pnpm db:migrate

# 7. Cargar datos de ejemplo (opcional)
pnpm db:seed

# 8. Iniciar el proyecto en modo desarrollo
pnpm dev
```

Acceder a la Aplicación

- **Frontend**: http://localhost:3001
- **Backend API**: http://localhost:3000
- **Prisma Studio**: http://localhost:5555 (ejecuta `pnpm db:studio`)

Credenciales por Defecto

Copiar

```
Email: admin@demo.com
Password: Admin123!
```

🎁 Stack Tecnológico

Backend

Tecnología	Versión	Propósito
Node.js	20.x	Runtime JavaScript
NestJS	10.x	Framework backend modular
TypeScript	5.x	Lenguaje tipado
Prisma	5.x	ORM para PostgreSQL
PostgreSQL	15.x	Base de datos relacional
Redis	7.x	Cache y colas de mensajes
Bull	4.x	Sistema de colas
Socket.io	4.x	WebSockets en tiempo real
Passport.js	0.7.x	Autenticación
JWT	9.x	Tokens de autenticación

Frontend

Tecnología	Versión	Propósito
Next.js	14.x	Framework React con SSR
React	18.x	Librería UI
TypeScript	5.x	Lenguaje tipado
Tailwind CSS	3.x	Framework CSS
Shadcn/ui	Latest	Componentes UI
Zustand	4.x	Estado global
React Query	5.x	Gestión de datos asíncronos

Tecnología	Versión	Propósito
React Hook Form	7.x	Formularios
Recharts	2.x	Gráficos y visualizaciones

Inteligencia Artificial

Tecnología	Propósito
OpenAI API	GPT-4 para generación de respuestas
LangChain	Orquestación de LLMs y memoria
Pinecone	Base de datos vectorial (RAG)
Tiktoken	Conteo de tokens

WhatsApp Integration

Tecnología	Propósito
@whiskeysockets/baileys	Conexión WhatsApp Web (QR)
whatsapp-web.js	Alternativa para conexión QR
Meta WhatsApp API	API oficial de WhatsApp Business

DevOps

Tecnología	Propósito
Docker	Contenedorización
Docker Compose	Orquestación local
GitHub Actions	CI/CD
Nginx	Reverse proxy
PM2	Process manager

📁 Estructura del Proyecto

Copiar

```
opentalkwisp/
├── apps/
│   ├── backend/                    # 🔧 API NestJS
│   │   ├── src/
│   │   │   ├── main.ts
│   │   │   ├── app.module.ts
│   │   │   ├── core/               # Lógica de dominio
│   │   │   ├── modules/            # Módulos funcionales
│   │   │   │   ├── auth/           # Autenticación
│   │   │   │   ├── contacts/       # Gestión de contactos
│   │   │   │   ├── conversations/  # Chats
│   │   │   │   ├── messages/       # Mensajes
│   │   │   │   ├── campaigns/      # Campañas
│   │   │   │   ├── pipeline/       # Pipeline de ventas
│   │   │   │   ├── flows/          # Flujos conversacionales
│   │   │   │   ├── ai/             # Servicios de IA
│   │   │   │   └── whatsapp/       # Proveedores WhatsApp
│   │   │   ├── infrastructure/     # DB, cache, APIs
│   │   │   └── shared/             # Utilidades compartidas
│   │   ├── prisma/
│   │   │   ├── schema.prisma       # Schema de base de datos
│   │   │   ├── migrations/         # Migraciones
│   │   │   └── seed.ts             # Datos iniciales
│   │   ├── test/                   # Tests
│   │   └── package.json
│   │
│   └── frontend/                   # 🎨 App Next.js
│       ├── src/
│       │   ├── app/                # App Router
│       │   │   ├── (auth)/         # Páginas de autenticación
│       │   │   └── (dashboard)/    # Páginas del dashboard
│       │   │       ├── contacts/
│       │   │       ├── conversations/
│       │   │       ├── pipeline/
│       │   │       ├── campaigns/
│       │   │       ├── flows/
│       │   │       └── analytics/
│       │   ├── components/         # Componentes React
│       │   │   ├── ui/             # Componentes base
│       │   │   ├── layout/         # Layout components
│       │   │   ├── contacts/
│       │   │   ├── chat/
│       │   │   └── pipeline/
│       │   ├── lib/                # Librerías y utilidades
│       │   │   ├── api/            # Cliente API
│       │   │   ├── hooks/          # Custom hooks
│       │   │   └── stores/         # Zustand stores
│       │   └── types/              # TypeScript types
│       └── package.json
│
├── packages/                       # 📦 Código compartido
│   ├── shared-types/               # Types compartidos
│   ├── shared-utils/               # Utilidades
│   └── config/                     # Configuraciones
│
├── docs/                           # 📚 Documentación
│   ├── README.md                   # Índice de documentación
│   ├── 01-PROJECT-BASE.md          # Documento base
```

```
│   ├── 02-DATABASE-DESIGN.md       # Diseño de BD
│   ├── 03-API-DOCUMENTATION.md     # API REST
│   ├── 04-AI-INTEGRATION.md        # Integración IA
│   ├── 05-WHATSAPP-PROVIDERS.md    # Proveedores WhatsApp
│   ├── 06-FRONTEND-ARCHITECTURE.md # Arquitectura Frontend
│   ├── 07-DEPLOYMENT-GUIDE.md      # Guía de despliegue
│   ├── 08-SECURITY-GUIDE.md        # Seguridad
│   ├── 09-TESTING-STRATEGY.md      # Testing
│   └── 10-CONTRIBUTING.md          # Contribución
│
├── docker/                         # 🐳 Docker
│   ├── Dockerfile.backend
│   ├── Dockerfile.frontend
│   └── docker-compose.yml
│
├── scripts/                        # 🔨 Scripts de utilidad
│   ├── setup.sh
│   ├── migrate.sh
│   └── deploy.sh
│
├── .github/
│   └── workflows/
│       └── ci-cd.yml               # GitHub Actions
│
├── .env.example                    # Variables de entorno ejemplo
├── .gitignore
├── package.json                    # Root package (monorepo)
├── pnpm-workspace.yaml             # Configuración workspaces
├── turbo.json                      # Turborepo config
├── LICENSE
└── README.md                       # 👈 Este archivo
```

🔧 Configuración

Variables de Entorno

Backend (`.env`)

Copiar

```
# Application
NODE_ENV=development
PORT=3000
APP_URL=http://localhost:3000
FRONTEND_URL=http://localhost:3001

# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/opentalkwisp?schema=public

# Redis
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=your-super-secret-jwt-key-change-this-in-production
```

```
JWT_EXPIRATION=7d
JWT_REFRESH_SECRET=your-refresh-secret-key
JWT_REFRESH_EXPIRATION=30d

# OpenAI
OPENAI_API_KEY=sk-your-openai-api-key-here
OPENAI_ORG_ID=org-your-org-id-here

# LangChain (opcional)
LANGCHAIN_API_KEY=your-langchain-key
LANGCHAIN_PROJECT=opentalkwisp

# Pinecone (opcional - para RAG)
PINECONE_API_KEY=your-pinecone-key
PINECONE_ENVIRONMENT=us-west1-gcp
PINECONE_INDEX=opentalkwisp-knowledge

# WhatsApp Meta Cloud API
META_APP_ID=your-meta-app-id
META_APP_SECRET=your-meta-app-secret
META_PHONE_NUMBER_ID=your-phone-number-id
META_ACCESS_TOKEN=your-access-token
META_WEBHOOK_VERIFY_TOKEN=your-webhook-verify-token

# Email (para notificaciones)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password

# Storage (AWS S3 para archivos multimedia)
AWS_ACCESS_KEY_ID=your-aws-access-key
AWS_SECRET_ACCESS_KEY=your-aws-secret-key
AWS_S3_BUCKET=opentalkwisp-media
AWS_REGION=us-east-1

# Encryption (para datos sensibles)
ENCRYPTION_KEY=your-32-byte-hex-encryption-key-here
```

Frontend (`.env.local`)

Copiar

```
NEXT_PUBLIC_API_URL=http://localhost:3000/api
NEXT_PUBLIC_WS_URL=ws://localhost:3000
NEXT_PUBLIC_APP_NAME=OpenTalkWisp
```

Docker Compose

El proyecto incluye un `docker-compose.yml` para desarrollo local:

Copiar

```
# Iniciar servicios (PostgreSQL + Redis)
docker-compose up -d

# Ver logs
docker-compose logs -f
```

```
# Detener servicios
docker-compose down

# Detener y eliminar volúmenes
docker-compose down -v
```

☐ Integración de IA

Configuración de OpenAI

1. **Obtener API Key**: https://platform.openai.com/api-keys
2. **Agregar a `.env`**:
   Copiar

```
OPENAI_API_KEY=sk-...
```

Funcionalidades de IA

1. Respuestas Automáticas

Copiar

```
// Ejemplo de uso en el código
const response = await aiService.generateResponse({
  conversationId: 'uuid',
  userMessage: 'Hola, necesito información sobre precios',
  systemPrompt: SYSTEM_PROMPTS.SALES_ASSISTANT,
});
```

2. Lead Scoring

Copiar

```
// Calcular score automáticamente
const score = await leadScoringService.calculateLeadScore('contact-uuid');
// Retorna: 0-100
```

3. Análisis de Sentimiento

Copiar

```
// Analizar sentimiento de un mensaje
const sentiment = await sentimentService.analyzeSentiment(
  'Estoy muy molesto con el servicio'
);
// Retorna: { sentiment: 'negative', score: -0.8, confidence: 0.95 }
```

4. Flujos con IA

Copiar

```
// Crear flujo desde descripción en lenguaje natural
const flow = await flowService.createWithAI(
  'Crea un flujo que salude, pregunte el nombre, y ofrezca 3 opciones'
);
```

Costos Estimados de IA

Operación	Tokens Promedio	Costo Aprox.
Respuesta simple	200	$0.002
Respuesta compleja	500	$0.005
Lead scoring	300	$0.003
Análisis sentimiento	100	$0.001

Costo mensual estimado (1000 conversaciones/día): **$150-300 USD**

Ver documentación completa: 📄 docs/04-AI-INTEGRATION.md

💬 Proveedores de WhatsApp

Opción 1: Meta Cloud API (Recomendado para Producción)

Ventajas:

- ✅ Oficial y estable
- ✅ Soporta plantillas aprobadas
- ✅ Mejor para envíos masivos
- ✅ Menos riesgo de bloqueo

Desventajas:

- ❌ Requiere aprobación de Meta
- ❌ Costos por mensaje ($0.005-0.009)
- ❌ Proceso de configuración más complejo

Configuración:

1. Crear app en [Meta for Developers]
2. Configurar WhatsApp Business API
3. Obtener credenciales
4. Agregar a `.env`:
   Copiar

   ```
   META_APP_ID=...
   META_APP_SECRET=...
   META_PHONE_NUMBER_ID=...
   META_ACCESS_TOKEN=...
   ```

Opción 2: Baileys (QR Code)

Ventajas:

- ✅ Gratis
- ✅ Configuración rápida
- ✅ Ideal para desarrollo y testing
- ✅ No requiere aprobación

Desventajas:

- ❌ No oficial (riesgo de bloqueo)
- ❌ Requiere escanear QR periódicamente
- ❌ Menos estable para producción
- ❌ No soporta plantillas oficiales

Configuración:

Copiar

```
// Se configura automáticamente
// Solo escanea el QR desde el dashboard
```

Patrón Adapter

El sistema usa el **Adapter Pattern** para cambiar entre proveedores sin modificar código:

Copiar

```
interface IWhatsAppProvider {
  connect(): Promise<void>;
  sendMessage(to: string, message: Message): Promise<void>;
  onMessage(callback: (msg: UnifiedMessage) => void): void;
```

```
}

// Implementaciones:
class MetaCloudProvider implements IWhatsAppProvider { }
class BaileysProvider implements IWhatsAppProvider { }
```

Ver documentación completa: 📄 docs/05-WHATSAPP-PROVIDERS.md

📊 Base de Datos

Tecnología

- **PostgreSQL 15+** con **Prisma ORM**
- **Redis 7+** para cache y colas

Modelos Principales

Copiar

```
Organizations (Multi-tenancy)
├── Users (Agentes, Admins)
├── WhatsAppInstances (Conexiones)
├── Contacts (Clientes)
│   ├── Conversations
│   │   └── Messages
│   ├── Deals (Oportunidades)
│   └── Notes
├── Campaigns
│   └── CampaignMessages
├── Flows (Flujos conversacionales)
│   └── FlowExecutions
├── Pipelines
│   └── PipelineStages
└── AiConfigs (Configuraciones de IA)
```

Comandos Útiles

Copiar

```
# Generar cliente de Prisma
pnpm db:generate

# Crear migración
pnpm db:migrate

# Aplicar migraciones
pnpm db:deploy

# Cargar datos de ejemplo
pnpm db:seed
```

```
# Abrir Prisma Studio (GUI)
pnpm db:studio

# Resetear base de datos (CUIDADO)
pnpm db:reset
```

Migraciones

Copiar

```
# Crear nueva migración
pnpm db:migrate --name add_user_preferences

# Ver estado de migraciones
pnpm db:migrate status

# Revertir última migración
pnpm db:migrate rollback
```

Ver documentación completa: 📄 docs/02-DATABASE-DESIGN.md

☐ Testing

Estrategia de Testing

Copiar

```
apps/backend/
├── test/
│   ├── unit/              # Tests unitarios
│   ├── integration/       # Tests de integración
│   └── e2e/              # Tests end-to-end
```

Comandos

Copiar

```
# Ejecutar todos los tests
pnpm test

# Tests en modo watch
pnpm test:watch

# Tests con coverage
pnpm test:cov

# Tests E2E
pnpm test:e2e

# Tests específicos
```

```
pnpm test contacts.service
```

Coverage Objetivo

- **Unit Tests**: 80%+
- **Integration Tests**: 70%+
- **E2E Tests**: Flujos críticos

Ejemplo de Test

Copiar

```
// contacts.service.spec.ts
describe('ContactsService', () => {
  it('should create a contact', async () => {
    const contact = await service.create({
      phoneNumber: '+5215512345678',
      name: 'Test User',
      organizationId: 'org-uuid',
    });

    expect(contact.phoneNumber).toBe('+5215512345678');
    expect(contact.name).toBe('Test User');
  });

  it('should prevent duplicate phone numbers', async () => {
    await service.create({
      phoneNumber: '+5215512345678',
      organizationId: 'org-uuid',
    });

    await expect(
      service.create({
        phoneNumber: '+5215512345678',
        organizationId: 'org-uuid',
      })
    ).rejects.toThrow();
  });
});
```

Ver documentación completa: 📄 docs/09-TESTING-STRATEGY.md

🚢 Despliegue

Opción 1: Docker (Recomendado)

Copiar

```
# Build de imágenes
```

```
docker-compose -f docker-compose.prod.yml build

# Iniciar servicios
docker-compose -f docker-compose.prod.yml up -d

# Ver logs
docker-compose -f docker-compose.prod.yml logs -f
```

Opción 2: VPS Manual

Copiar

```
# 1. Instalar dependencias en el servidor
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs postgresql redis-server

# 2. Clonar y configurar
git clone https://github.com/tu-usuario/opentalkwisp.git
cd opentalkwisp
pnpm install
pnpm build

# 3. Configurar PM2
pm2 start ecosystem.config.js
pm2 save
pm2 startup
```

Opción 3: Cloud Providers

AWS

- **EC2** para aplicación
- **RDS PostgreSQL** para base de datos
- **ElastiCache Redis** para cache
- **S3** para archivos multimedia
- **CloudFront** para CDN

DigitalOcean

- **Droplet** para aplicación
- **Managed PostgreSQL** para base de datos
- **Managed Redis** para cache
- **Spaces** para archivos

Vercel (Solo Frontend)

Copiar

```
# Deploy frontend a Vercel
cd apps/frontend
vercel deploy --prod
```

Variables de Entorno en Producción

Copiar

```
NODE_ENV=production
DATABASE_URL=postgresql://user:pass@prod-db:5432/opentalkwisp
REDIS_URL=redis://prod-redis:6379
# ... resto de variables
```

Ver documentación completa: 📄 docs/07-DEPLOYMENT-GUIDE.md

📚 Documentación Completa

Documentos Principales

Documento	Descripción
📖 Índice de Documentación	Punto de entrada a toda la documentación
📄 01-PROJECT-BASE.md	Visión general, objetivos, stack tecnológico
📄 02-DATABASE-DESIGN.md	Schema completo de Prisma, relaciones, migraciones
📄 03-API-DOCUMENTATION.md	Endpoints REST, autenticación, ejemplos
📄 04-AI-INTEGRATION.md	OpenAI, LangChain, flujos, prompts
📄 05-WHATSAPP-PROVIDERS.md	Meta API, Baileys, patrón Adapter
📄 06-FRONTEND-ARCHITECTURE.md	Next.js, componentes, estado, diseño
📄 07-DEPLOYMENT-GUIDE.md	Docker, Kubernetes, Cloud, CI/CD
📄 08-SECURITY-GUIDE.md	Autenticación, encriptación, GDPR
📄 09-TESTING-STRATEGY.md	Unit, integration, E2E tests
📄 10-CONTRIBUTING.md	Cómo contribuir al proyecto

Guías Rápidas

- 🚀 Inicio Rápido
- 💻 Desarrollo Local
- 🔧 Solución de Problemas
- ❓ FAQ

☐ Contribuir

¡Las contribuciones son bienvenidas! Por favor lee nuestra Guía de Contribución antes de enviar un Pull Request.

Proceso de Contribución

1. **Fork** el repositorio
2. **Crea** una rama para tu feature (`git checkout -b feature/AmazingFeature`)
3. **Commit** tus cambios (`git commit -m 'Add some AmazingFeature'`)
4. **Push** a la rama (`git push origin feature/AmazingFeature`)
5. **Abre** un Pull Request

Estándares de Código

- **ESLint** para linting
- **Prettier** para formateo
- **Conventional Commits** para mensajes de commit
- **TypeScript strict mode**

Copiar

```
# Verificar código
pnpm lint

# Formatear código
pnpm format

# Ejecutar tests antes de commit
pnpm test
```

Reportar Bugs

Usa GitHub Issues con la plantilla de bug report.

Solicitar Features

Usa GitHub Issues con la plantilla de feature request.

📊 Roadmap

✅ Fase 1: MVP (Completado)

- ☑ Autenticación y autorización
- ☑ CRUD de contactos
- ☑ Chat en tiempo real
- ☑ Conexión WhatsApp (QR)
- ☑ Envío/recepción de mensajes

🚧 Fase 2: CRM Core (En Progreso)

- ☑ Pipeline de ventas
- ☑ Etiquetado de contactos
- ☐ Asignación automática de conversaciones
- ☐ Reportes básicos

📅 Fase 3: IA Básica (Q1 2026)

- ☐ Respuestas automáticas
- ☐ Análisis de sentimiento
- ☐ Lead scoring automático

📅 Fase 4: Campañas (Q2 2026)

- ☐ Broadcasting masivo
- ☐ Plantillas de mensajes
- ☐ Programación de envíos
- ☐ Métricas de campaña

📅 Fase 5: IA Avanzada (Q3 2026)

- ☐ LangChain integration
- ☐ Constructor de flujos con IA
- ☐ RAG para conocimiento empresarial
- ☐ Chatbots personalizados

📅 Fase 6: Omnicanalidad (Q4 2026)

- ☐ Facebook Messenger
- ☐ Instagram DM
- ☐ Telegram
- ☐ Bandeja unificada

📄 Licencia

Este proyecto está bajo la licencia **MIT**. Ver el archivo LICENSE para más detalles.

Copiar

```
MIT License

Copyright (c) 2025 OpenTalkWisp

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

👥 Equipo

Core Team

- **[Tu Nombre]** - *Lead Developer* - @tu-github

Contribuidores

Ver la lista completa de contribuidores que participaron en este proyecto.

🙏 Agradecimientos

- NestJS - Framework backend increíble
- Next.js - El mejor framework React
- Prisma - ORM moderno y potente
- OpenAI - Tecnología de IA de vanguardia
- Baileys - Librería WhatsApp Web
- Shadcn/ui - Componentes UI hermosos

📞 Soporte y Contacto

Canales de Soporte

- 📧 **Email**: support@opentalkwisp.com
- 💬 **Discord**: Unirse al servidor
- 🐛 **Issues**: GitHub Issues
- 📖 **Documentación**: docs.opentalkwisp.com
- 🌐 **Website**: opentalkwisp.com

Redes Sociales

- **Twitter**: @opentalkwisp
- **LinkedIn**: OpenTalkWisp

- **YouTube**: Canal de OpenTalkWisp

📈 Estado del Proyecto

⭑ **Si este proyecto te resulta útil, considera darle una estrella en GitHub** ⭑

Hecho con ❤️ por el equipo de OpenTalkWisp

⬆️ Volver arriba

Última actualización: Diciembre 2025
Versión: 1.0.0